Data Analysis Report
Scrum Game AI Opponent Project

Deliverable A – Dataset Analysis Report

Course / project	**Big Data & AI - Scrum Game NPC**
Group members	Jochem Schippers, Mira Philippo, Sem Philippo, Damisa Vooges
Client	Witan Entertainment
Date	22 March 2026

Note: this project does not have a "normal" data set since our project differs a bit from the others. The purpose of this report is to analyse the available game rules and configuration variables and translate them into a structured dataset design that can later be used for simulation and machine learning.

1. Introduction

Our project is about developing an AI opponent for the Scrum Game. Compared to other groups on this analysis, our document will be differently structured because we currently have a rule-based game description, an example of the physical board, and a list of variables that can be configured in the simulator but not a big dataset to gather data from.

Because of that, we had to take a bit of a different approach. Instead of pretending that we already have a complete gameplay dataset and cleaning up data just for the sake of cleaning it, we focus on the part that is more important for this part of the project, which is defining which variables matter, how they should be stored, what the future model should learn from them, and which checks are needed before simulation data can be generated.

The main goal of this report is to prepare a dataset structure for the AI to use. Since we will use reinforcement learning, the model will not simply predict a fixed label from old examples. Instead, it will learn from repeated game situations, actions, and rewards generated by the simulator.

2. Problem Description and Project Goal

In the Scrum Game, players constantly make decisions without knowing exactly what will happen. Players choose products, complete sprints, deal with risk from dice rolls, pay switching costs, and can be forced to take loans when money runs out. A good AI opponent should therefore be able to reason about both short and long-term effects. It should not only react to the current turn, but also consider future value, risk, debt pressure, and the limited number of remaining turns.

A rule-based bot would become too predictable and would not adapt well to changing situations. For that reason, we chose to build a simulator first. This simulator will act as the environment in which the AI can train through many repeated games and learn which actions lead to better outcomes. This fits the project well, because the goal is to create an adaptive AI opponent for the Scrum Game.

3. Source Description

The information used for this report comes from project documentation rather than from collected gameplay logs. The sources mainly describe how the game works, which values are used on the board, and which simulation settings can be changed.

Source	Type of information	Use in this report
Scrum Game manual	Game rules	Used to understand turn flow, dice mechanics, sprint success and failure, penalties, loans, and legal actions
Physical example game	Board layout and default values	Used to identify products, sprints, value differences, and feature differences between board positions

Simulation variables setup	Configurable variables	Used to define which parameters can become part of the generated dataset and the simulator state
Data Analysis checklist	Assignment structure	Used to make sure the report still covers the expected school requirements, with explanation where the standard checklist is not fully applicable

3.1 Dataset source and collection method

At the moment, we don't have a dataset collected from finished games. So, the current 'dataset' is the game documentation and rules itself. The data is gathered by document analysis: reading the rules, extracting variables, and converting them into a structure that can later be used by the simulator.

3.2 Type of data

The future dataset will mostly be structured data. It will contain numerical values such as money, sprint value, number of features, switching cost, and debt. It will also contain categorical variables such as current product, and strategy profile. Some descriptive text fields may exist in the documentation, but those are not important as direct model inputs.

3.3 Why this is not a standard dataset yet

Normally this section would state the number of rows and columns in a CSV or database table. We cannot do that honestly yet, because the actual rows will only exist after the simulator starts logging game states. What we can do already is define what one future row should look like. In this project, one row can be seen as one recorded game state or one state-action-result step. So we are also not sure yet how much data we'll eventually end up with.

4. Proposed Dataset Structure

To make the project workable, the game should be translated into a few main entities. This helps both the simulator and the later AI model.

Entity	Example variables	Data type	Purpose
Game configuration	number_of_players, number_of_products, sprints_per_product, starting_money, switch_cost	Numerical / categorical	Defines the environment for one match
Board cell	product_id, sprint_id, sprint_value, features_required	Mixed	Represents risk and reward of a specific sprint
Player	money, current_product,	Mixed	Represents the current

state	current_sprint, loan_active, interest_due		situation of the agent
Round effects	changed_value, refinement_effect	Mixed	Captures events that change future decisions
Action	continue, switch_product, chosen_product, take_loan	Categorical	Stores what the agent decided
Outcome	sprint_success, payout, penalty, final_money, reward	Numerical / boolean	Stores the result of the action

4.1 Feature overview

Feature	Type	Description	Role
current_money	Numerical	Current amount of money available to the player.	State
current_product	Categorical	Which product the player is currently developing.	State
current_sprint	Numerical	Sprint position within the chosen product.	State
features_required	Numerical	Number of features that determines dice complexity and risk.	State
sprint_value	Numerical	Potential financial value if the sprint succeeds.	State
switch_cost	Numerical	Cost paid when changing to another product.	Environment
loan_active	Boolean	Whether the player currently has a loan running.	State
interest_due	Numerical	Interest to be paid on future turns if a loan is active.	State
reward	Numerical	Learning signal based on result of the action.	Training target

4.2 Example of one future data row

An example recorded row could look like this: turn = 7, current_money = 17000, current_product = blue, current_sprint = 2, features_required = 4, sprint_value = 6, loan_active = no, payout = -5000, penalty = 0, reward = -3. A row like this is much closer to what the AI actually needs than a traditional company spreadsheet because it captures the decision state, the action, and the result.

5. Exploratory Analysis of the Available Information

Since there is no gameplay log yet, the exploratory analysis focuses mainly on ranges of the variables, the logical relationships, and the risk-reward patterns that can already be seen from the game design.

5.1 Important simulator variables and default settings

The table below is based mainly on the simulator configuration document. The "Default value" column shows the default simulator setting, while the "Possible range" column shows the allowed configuration range for generated games.

Simulator Variable	Default value	Possible range	Why it matters for the model
Number of players	4	1-12	Higher numbers increase interaction and complexity
Number of products	7	4-12	More products create more choices and more switching decisions
Sprints per product	4	4-12	Determines planning horizon and board depth
Game length (sprints per player)	6-20 sprints per player (6 advised)	4-32	Changes urgency and long-term planning
Starting money	EUR 25,000	0-250,000	Affects early-game risk tolerance
Switch cost	EUR 5,000 / EUR 0 after last sprint	0-250,000	Discourages changing products too easily
Loan amount	EUR 50,000	0-250,000	Acts as a recovery mechanic when money runs out
Loan interest	EUR 5,000	0-250,000	Creates long-term pressure after borrowing
Daily scrum	12	4-12	Changes failure rate and

target			sprint difficulty

5.2 Patterns that already matter

A first important pattern is that higher sprint value is not automatically better. A sprint with a high value can also require more features, which means more difficult dice conditions and a larger chance of failure. So the AI should not only look at value, but also at the risk attached to that value.

A second pattern is the trade-off around switching. Switching can be useful if another product has a better expected outcome, but it immediately costs money and may reset progress. This means the AI needs to estimate whether the long-term gain is worth the short-term loss.

A third pattern is debt pressure. Loans prevent bankruptcy, but the interest can slowly destroy future profit. That makes money management a core part of the state representation.

5.3 What we cannot calculate yet

Things like mean, median, standard deviation, class distribution, and correlation heatmaps are not useful and able to be made yet, because there is no dataset available yet. For this deliverable, it made more sense for us to focus on the variables in the game, how they are connected, and what relationships we expect between them.

6. Dataset Visualisation Approach

The assignment checklist asks for visualisations such as histograms, bar charts, and correlation plots. At this moment, we are not really able to provide these since we don't have strict data like that yet. That said, we included some non-statistical visuals in this part.

Visual	Use now?	Reason
Board overview table	Yes	Shows products, sprint values, and feature counts from the default board
State-action diagram	Yes	Explains how one game state becomes an action and then a reward
Histogram of values	Later	Useful only after many simulated rows exist
Bar chart of actions	Later	Useful after logging action frequencies from the AI or baseline agents
Correlation heatmap	Later	Useful after numeric simulation data has been generated

7. Data Quality Analysis

Even though we do not yet have raw dirty data, data quality is still important. In this project, quality mainly means that the rule model is complete and consistent. If the simulator contains errors, the generated training data will also be wrong.

Quality check	Why it matters
Every board cell has a valid value and feature count	Without this, the simulator cannot calculate legal risk and reward
Money can never become undefined	Loan logic, penalties, and reward all depend on valid money values
Switching rules are implemented consistently	The AI must know which actions are legal and what they cost
Refinement only affects future sprints	This is required by the rule setup and changes state transitions
Incident effects are applied consistently	Incorrect event handling would create misleading training data
Penalty logic is stable	Small mistakes here can strongly distort learned behaviour

7.1 Missing values, outliers, and inconsistencies

We can't make a missing-value analysis because the current source is documentation, not a spreadsheet with empty cells. Still, there is an equivalent issue: incomplete rule definitions. If one product is missing values or one action has no defined result, that behaves like a missing value inside the simulator.

Outliers are also different here. We do not yet have abnormal observations from real play, but we can still create extreme settings such as very high penalties, very low switch cost, or very long games. These settings should not be mixed into baseline experiments without being labelled clearly.

Duplicates and inconsistent labels should mainly be controlled at implementation level. For example, product names, action names, and incident names should follow one consistent naming scheme so they can be encoded correctly later.

7.2 Possible bias

The biggest likely bias in this project is simulator bias. The AI can only learn patterns that the simulator exposes. If the reward function overvalues short-term money, the model may become too greedy. If only one board configuration is used, the model may overfit that setup. To reduce this risk, the simulator should eventually support variation in board settings, incidents, and strategy profiles.

8. Data Cleaning and Preprocessing

For this project, preprocessing mostly means converting the rules into a stable numeric representation that machine learning can use. This is less about cleaning messy human-entered data, and more about making sure the simulator outputs clean and consistent records.

Step	Needed?	Explanation
Missing value handling	Partly	Not needed for the documents themselves, but every generated state should be complete before training
Data type correction	Yes	Money, counts, flags, and rewards should use consistent numeric or boolean formats
Outlier handling	Partly	Extreme configurations should be tested separately instead of being treated as normal baseline data
Encoding categorical variables	Yes	Product ID and strategy profile must be encoded for the model
Scaling / normalisation	Likely yes	Financial variables may otherwise dominate smaller state features
Feature engineering	Yes	Useful derived variables can summarize risk, urgency, and debt pressure
Feature selection	Yes	Only variables that actually affect decisions should stay in the final training input

8.1 Possible engineered features

Possible engineered features include remaining turns, debt pressure ratio, expected sprint risk, estimated future value of the current product, and whether switching would reset progress. These features combine several variables into more meaningful signals. That can make learning easier than only feeding raw values into the model.

8.2 Explanation of removed data

Some variables may be left out of model training even if they appear in the documentation. Examples are descriptive product names, human-readable card text, and other fields that do not directly influence legal actions or payoffs. These could still remain in logs for readability, but they do not need to become model features.

9. Feature Selection and Model Preparation

9.1 Input features (X)

The input vector should describe the state of the game at the moment the AI has to make a decision. A practical first version of X could include current money, current product, current sprint, remaining turns, switch cost, loan status, interest due, active incident effects, next sprint values, and next sprint feature requirements.

9.2 Target variable / output (y)

For a standard supervised model, y would be a target label such as the best action. For this project, reinforcement learning is more suitable. In that case, the model output is the chosen action or policy decision in each state. Examples are continuing current product, switching to another product, or any other legal move supported by the simulator.

9.3 Reward signal

The reward design is an important part of the project. A reasonable starting point is to base reward on financial change after a move, while also penalizing failed sprints, unnecessary debt, or risky actions that create long-term problems. An end-of-game bonus can be added for winning or finishing with the highest amount of money.

9.4 Why these features are used

These features are used because they directly influence whether a decision is good or bad. Money and debt show financial health. Sprint values and feature counts show risk-reward trade-offs. Remaining turns show urgency. Incidents and refinement effects make the environment dynamic. Together, these variables give the model enough information to make strategy-based decisions instead of random guesses.

10. Best Project Approach Going Forward

Based on the project setup, the best approach is not to jump straight into model training. That would be a mistake, because the AI would then depend on an environment that is not yet optimized.

Phase	Main task	Why this comes first
1	Formalise rules and variables	Without a correct state and action model, every later result is unreliable
2	Build a headless simulator	The simulator is needed to generate many playable situations quickly
3	Log state-action-reward data	This creates the real dataset that can finally be analysed statistically
4	Run baseline agents	Simple strategies create comparison points

		before training the full AI
5	Train and evaluate RL agent	Only after the environment and logging are stable does training become meaningful

This order is important because it prevents wasted time. If the simulator is wrong, the model may learn the wrong lessons very efficiently, which could become bad quickly.

11. Checklist Items That Are Only Partly Applicable

Checklist item	How to explain it in this project
Number of rows and columns	There is no final gameplay dataset yet. Instead, this report defines the future simulation schema and what one row should represent
Mean / median / standard deviation	These can only be calculated after enough simulated or observed matches have been logged
Distribution plots / boxplots	Useful later, but not meaningful yet without generated rows
Class distribution	Only relevant if labelled actions or outcomes already exist
Missing values	Reframed here as incomplete rule definitions or incomplete simulator states
Outliers	Reframed here as extreme parameter settings that should be tested separately

12. Conclusion

The main conclusion of this report is that the Scrum Game project should not be treated as a normal dataset-cleaning task yet. At this phase, the real data-analysis work is to define a correct and useful simulation dataset structure. This includes identifying the important variables, the allowed ranges, the state representation, the legal actions, and the reward logic.

That is why this project is better suited to reinforcement learning in a simulated environment. For the next deliverable, our focus should be on building and testing the simulator and using it for the first version of the model. This will then be explained in the model report and linked to the code implementation. After enough gameplay data has been generated, we can later add more standard data analysis like summary statistics, correlations, action distributions, and visualisations.

13. Individual Contributions

Author	Contribution

Jochem Schippers	Worked on the parts about the game rules, variables, and checking the report
Mira Philippo	Worked on the report structure, the source analysis, and checking if the content matched the project goals
Sem Philippo	Worked on analyzing the source documents and describing the important variables and features for the model
Damisa Vooges	Worked on the parts about the project approach, model setup, reward logic, and the link between the simulator and AI

14. References

Plaat, A. (2020). Learning to play: AI and games. Springer.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

Van der Meer, M. (2025). Scrum Game manual. Witan Entertainment.

Van der Meer, M. (2025). Physically boardgame - example game step by step. Witan Entertainment.

Van der Meer, M. (2026). Simulation variables to setup per game. Witan Entertainment.